DOCUMENTACIÓN PROYECTO PRUEBAS SABER



NOMBRES Y APELLIDOS DE LOS ESTUDIANTES

JOHAN SANTIAGO LOPEZ ORTEGA

SEBASTIAN TINTINAGO PANTOJA

EDDY EMMANUEL SANCHEZ OBANDO

Profesor:

Julio Ariel Hurtado Alegria

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Departamento de Sistemas

Ingeniería del *Software* II

Popayán, octubre 2026

JOHAN SANTIAGO LOPEZ ORTEGA

SEBASTIAN TINTINAGO PANTOJA

EDDY EMMANUEL SANCHEZ OBANDO

DOCUMENTACIÓN DE ARQUITECTURA DEL SISTEMA BANCO DE PREGUNTAS SABER PRO

Entregable del primer corte presentado en el curso de Ingeniería de Software II

Estudiantes del:

Programa de Ingeniería de Sistemas

Profesor:

Julio Ariel Hurtado Alegria

Popayán

2026

Contenido

Lista de Figuras iii
Lista de Tablas iv
Lista de Siglas v
Documentación de arquitectura del sistema Banco de Preguntas Saber Pro 1
1.- Introducción 1
2.- Historias de usuario implementadas 2
2.1.- HU-01: Crear pregunta de selección múltiple 2
2.2.- HU-02: Enviar pregunta a revisión 3
2.3.- HU-03: Listar mis preguntas con filtros y paginación 4
2.4.- HU-04: Asignar revisores 5
3.- Prototipos de la interfaz y test de usabilidad 6
3.1.- Prototipos de la interfaz de usuario 6
3.2.- Test de usabilidad (thinking aloud) 9
4.- Planificación del Sprint 1 10
5.- Requisitos no funcionales: escenarios de calidad 12
5.1.- Escenario 1: nuevo canal de notificación 12
5.2.- Escenario 2: nuevo tipo de pregunta 13
5.3.- Escenario 3: cambio del motor de base de datos 13
6.- Decisiones de arquitectura 14
6.1.- Tipo de aplicación: escritorio 14
6.2.- Estilo arquitectónico: monolito en tres capas con MVC 14
6.3.- Selección de tecnologías 15
7.- Arquitectura del sistema 17
7.1.- Nivel 1: diagrama de contexto 17
7.2.- Nivel 2: diagrama de contenedores 18
7.3.- Nivel 3: diagrama de componentes 19
7.4.- Nivel 4: diagramas de clases 21
7.5.- Vista dinámica: diagramas de secuencia 24
8.- Modelo de datos 27
9.- Principios SOLID y patrones de diseño 29
9.1.- Principios SOLID 29
9.2.- Patrones de diseño implementados 29
9.3.- Oportunidades de mejora para el segundo corte 33

10.- Pruebas unitarias automatizadas 34
11.- Enlaces del proyecto 36
Bibliografía 37

Lista de Figuras

Figura 3.1 Prototipo de la ventana de inicio de sesión 6
Figura 3.2 Prototipo del tablero por rol: autor de preguntas (izquierda) y administrador (derecha) 6
Figura 3.3 Prototipo del formulario «Nueva pregunta» (HU-01) 7
Figura 3.4 Prototipo de la ventana «Mis preguntas» con filtros, estados con color y paginación (HU-02 y HU-03) 8
Figura 3.5 Prototipo de la ventana «Asignar revisores» (HU-04) 8
Figura 4.1 Tablero de tareas del Sprint 1 [reemplazar por el pantallazo de Trello o Jira] .. 10
Figura 7.1 Diagrama de contexto (C4, nivel 1) 17
Figura 7.2 Diagrama de contenedores (C4, nivel 2) 19
Figura 7.3 Diagrama de componentes de la aplicación de escritorio (C4, nivel 3) 20
Figura 7.4 Diagrama de clases del dominio 22
Figura 7.5 Diagrama de clases de la presentación, los controladores y los servicios (MVC y Observer) 23
Figura 7.6 Diagrama de clases de la inversión de dependencias entre servicios, interfaces y repositorios 24
Figura 7.7 Diagrama de secuencia de HU-01: crear pregunta 25
Figura 7.8 Diagrama de secuencia de HU-04: asignar revisores 26
Figura 8.1 Modelo entidad-relación del sistema Banco de Preguntas Saber Pro 27
Figura 9.1 Patrón Factory para crear el notificador de revisores 31
Figura 9.2 Microkernel de preguntas con tuberías y filtros 32

Lista de Tablas

Tabla 2.1 Historia de usuario HU-01: Crear pregunta de selección múltiple 2
Tabla 2.2 Historia de usuario HU-02: Enviar pregunta a revisión 3
Tabla 2.3 Historia de usuario HU-03: Listar mis preguntas con filtros y paginación 4
Tabla 2.4 Historia de usuario HU-04: Asignar revisores 5
Tabla 3.1 Tareas del test de usabilidad 9
Tabla 4.1 Tareas del Sprint 1 11
Tabla 5.1 Escenario de calidad de modificabilidad 1 12
Tabla 5.2 Escenario de calidad de modificabilidad 2 13
Tabla 5.3 Escenario de calidad de modificabilidad 3 13
Tabla 6.1 Tecnologías seleccionadas 15
Tabla 7.1 Paquetes del código y su responsabilidad 21
Tabla 8.1 Tablas implementadas en el primer corte y su correspondencia con el modelo 28
Tabla 9.1 Aplicación de los principios SOLID 29
Tabla 9.2 Patrones de diseño implementados 30
Tabla 10.1 Pruebas unitarias automatizadas 34

Lista de Siglas

Sigla	Significado
C4	Modelo de contexto, contenedores, componentes y código
DIP	Principio de inversión de dependencias (Dependency Inversion Principle)
ER	Entidad-relación
GoF	Gang of Four (autores del libro Design Patterns)
GUI	Interfaz gráfica de usuario
HU	Historia de usuario
ISP	Principio de segregación de interfaces
JDBC	Java Database Connectivity
LSP	Principio de sustitución de Liskov
MVC	Modelo-Vista-Controlador
OCP	Principio abierto/cerrado
RF	Requisito funcional
RNF	Requisito no funcional
SMTP	Simple Mail Transfer Protocol
SOLID	SRP, OCP, LSP, ISP y DIP
SQL	Structured Query Language
SRP	Principio de responsabilidad única
TLS	Transport Layer Security
UML	Unified Modeling Language

Documentación de arquitectura del sistema Banco de Preguntas Saber Pro

1.- Introducción

Este documento describe la arquitectura y el diseño de la primera iteración (primer corte) del *Sistema para la Gestión, Validación y Administración de un Banco de Preguntas para la Preparación de las Pruebas Saber Pro*, desarrollado en el curso Ingeniería de Software II de la Universidad del Cauca [10].

En esta iteración se construyó una aplicación de escritorio en Java con una arquitectura monolítica en tres capas y el micropatrón Modelo-Vista-Controlador (MVC). Se implementaron las cuatro historias de usuario de alto valor definidas para el corte: crear preguntas con validación estructural, enviarlas a revisión, listarlas con filtros y paginación, y asignar revisores con notificación por correo. La aplicación conserva además el registro y la autenticación de usuarios por rol construidos en los talleres de laboratorio, y el generador de preguntas basado en los estilos Microkernel y Tuberías y filtros.

El documento se organiza así:

- El capítulo 2 presenta las historias de usuario implementadas y sus criterios de aceptación.
- El capítulo 3 muestra los prototipos de la interfaz y el protocolo del test de usabilidad.
- El capítulo 4 resume la planificación del Sprint 1.
- El capítulo 5 especifica los escenarios de calidad de modificabilidad.
- El capítulo 6 justifica el tipo de aplicación, el estilo arquitectónico y las tecnologías.
- El capítulo 7 documenta la arquitectura con el modelo C4 y diagramas UML.
- El capítulo 8 presenta el modelo entidad-relación.
- El capítulo 9 explica los principios SOLID y los patrones de diseño aplicados.
- El capítulo 10 describe las pruebas unitarias y el capítulo 11 reúne los enlaces del proyecto.

2.- Historias de usuario implementadas

Las historias de usuario siguen el formato «Yo como … necesito … para …». Los criterios de aceptación se redactaron con la estructura Dado/Cuando/Entonces y corresponden a reglas que el código valida y que las pruebas unitarias verifican (capítulo 10). Todas tienen prioridad alta porque son las historias de mayor valor definidas por el cliente para el primer corte.

2.1.- HU-01: Crear pregunta de selección múltiple

Rol	Autor de preguntas
Historia	Yo como autor de preguntas necesito crear preguntas de selección múltiple con única respuesta acorde a los principios del Diseño Centrado en Evidencia para alimentar el banco de preguntas Saber Pro.
Prioridad	Alta
Requisitos	RF-04, RF-05 y RF-08 a RF-11 (HU02 y HU03 del enunciado).
Criterios de aceptación	1. Dado que el autor diligencia título, contexto, pregunta directa, cuatro opciones (A a D), respuesta correcta, justificación, bibliografía, competencia, tema, subtema y nivel de dificultad, cuando pulsa «Guardar», entonces la pregunta se guarda en estado «Borrador», con un identificador consecutivo (P-001, P-002, …) y con el autor que inició sesión. 2. Dado que falta un campo obligatorio, cuando pulsa «Guardar», entonces el sistema no guarda la pregunta y muestra todos los errores juntos. 3. Dado que el contexto, la pregunta directa o la justificación tienen menos de 10 caracteres, o el título supera 100 caracteres, entonces el sistema rechaza la pregunta. 4. Dado que dos opciones tienen el mismo texto (sin distinguir mayúsculas), entonces el sistema indica cuál opción está repetida. 5. La respuesta correcta solo puede ser A, B, C o D, y el nivel de dificultad solo Fácil, Media o Difícil. 6. Solo el autor puede editar su pregunta, y únicamente mientras está en «Borrador»; al editar se vuelve a aplicar la validación estructural.
Clases que la implementan	GUIQuestionForm, QuestionController.crear(), QuestionService.createQuestion(), QuestionValidator.

Tabla 2.1 Historia de usuario HU-01: Crear pregunta de selección múltiple

2.2.- HU-02: Enviar pregunta a revisión

Rol	Autor de preguntas
Historia	Yo como autor de preguntas necesito cambiar el estado de mis preguntas de «Borrador» a «Pendiente de revisión» para que el administrador les asigne al menos un revisor.
Prioridad	Alta
Requisitos	RF-14 y RF-15 (transición Borrador → Pendiente de revisión).
Criterios de aceptación	1. Dado una pregunta propia en «Borrador», cuando el autor la selecciona en «Mis preguntas», pulsa «Enviar a revisión» y confirma, entonces su estado cambia a «Pendiente de revisión». 2. Antes del envío el sistema pide confirmación, porque después la pregunta ya no se puede editar. 3. Dado que la pregunta no está en «Borrador» o pertenece a otro autor, entonces el sistema rechaza la operación con un mensaje claro. 4. Los estados se muestran con colores: azul para «Borrador», ámbar para «Pendiente de revisión» y rojo para «Eliminada». 5. Al cambiar el estado, las ventanas abiertas (Mis preguntas, Asignar revisores y estadísticas) se actualizan solas.
Clases que la implementan	GUIMyQuestions, EstadoColores, QuestionController.enviarARevision(), QuestionService.sendToReview(), patrón Observer.

Tabla 2.2 Historia de usuario HU-02: Enviar pregunta a revisión

2.3.- HU-03: Listar mis preguntas con filtros y paginación

Rol	Autor de preguntas
Historia	Yo como autor de preguntas necesito listar las preguntas que he creado para poder más adelante verlas y editarlas.
Prioridad	Alta
Requisitos	RF-07.
Criterios de aceptación	1. El listado muestra solo las preguntas del autor que inició sesión, con id, título, tema, competencia, dificultad y estado. 2. Se puede filtrar por estado y dificultad (listas desplegables), por tema y competencia (texto parcial, sin distinguir mayúsculas) y por un texto que se busca en el título, el contexto y la pregunta. Los filtros se pueden combinar y el botón «Limpiar» los borra. 3. El listado se pagina con 5, 10 o 20 filas por página (10 por defecto), con los botones «< Anterior» y «Siguiente >» y el texto «Página X de Y (N preguntas)». 4. Si se pide una página fuera de rango, el sistema muestra la página válida más cercana. 5. Desde el listado se abre la pregunta seleccionada: si está en «Borrador» se puede editar; en otro estado solo se puede ver.
Clases que la implementan	GUIMyQuestions, QuestionFilter, PageResult<T>, QuestionService.searchByAuthor().

Tabla 2.3 Historia de usuario HU-03: Listar mis preguntas con filtros y paginación

2.4.- HU-04: Asignar revisores

Rol	Administrador
Historia	Yo como administrador necesito asignar al menos un revisor a las preguntas en estado «Pendiente de revisión» para que sean revisadas por otros docentes. Una vez asignados, el sistema envía un correo para notificarlos.
Prioridad	Alta
Requisitos	RF-16.
Criterios de aceptación	1. La ventana lista solo las preguntas en «Pendiente de revisión» y solo los usuarios activos con rol Revisor; también muestra los revisores ya asignados a la pregunta seleccionada. 2. El administrador puede seleccionar uno o varios revisores (Ctrl) y debe elegir al menos uno. 3. No se puede asignar al autor de la pregunta ni a usuarios inactivos o sin rol Revisor. Si algún revisor no es válido, no se guarda ninguna asignación. 4. Si un revisor ya estaba asignado, no se duplica la asignación ni se reenvía el correo. 5. Cada revisor nuevo recibe un correo con el id, el título, el tema, la competencia y el autor de la pregunta. Si no existe el archivo mail.properties, el correo se simula por consola. 6. Si falla el envío de un correo, la asignación se conserva y el sistema informa qué correos fallaron. 7. Solo un usuario con rol Administrador puede asignar revisores.
Clases que la implementan	GUIAssignReviewers, ReviewController, ReviewService.assignReviewers(), INotificador, NotificadorFactory.

Tabla 2.4 Historia de usuario HU-04: Asignar revisores

Alcance pendiente. La validación estructural cubre RF-08 a RF-11. Las reglas RF-12 (impedir expresiones como «Todas las anteriores» o «Ninguna de las anteriores») y RF-13 (longitud y estructura gramatical de los distractores) quedan planeadas para la siguiente iteración, como nuevas reglas de QuestionValidator o nuevos filtros del pipeline.

3.- Prototipos de la interfaz y test de usabilidad

3.1.- Prototipos de la interfaz de usuario

Los prototipos se elaboraron como wireframes y luego se implementaron con Java Swing, por lo que corresponden a las ventanas reales de la aplicación. Los datos que aparecen son de ejemplo.

La Figura 3.1 muestra el inicio de sesión. El usuario ingresa su login y su contraseña; desde la misma ventana puede abrir el formulario de registro.



Figura 3.1 Prototipo de la ventana de inicio de sesión

Después de autenticarse, el tablero muestra solo las opciones del rol del usuario (Figura 3.2). El autor ve «Crear pregunta» y «Mis preguntas»; el administrador ve «Asignar revisores». El revisor ve el banco de preguntas, y el docente y el estudiante ven un aviso de las funciones de los próximos cortes.



Figura 3.2 Prototipo del tablero por rol: autor de preguntas (izquierda) y administrador (derecha)

La Figura 3.3 corresponde a HU-01. El formulario pide los diez elementos de una pregunta Saber Pro; la respuesta correcta y el nivel de dificultad se eligen de listas para evitar errores de digitación. Al pulsar «Guardar» se aplica la validación estructural y, si hay errores, se muestran todos en un solo mensaje.

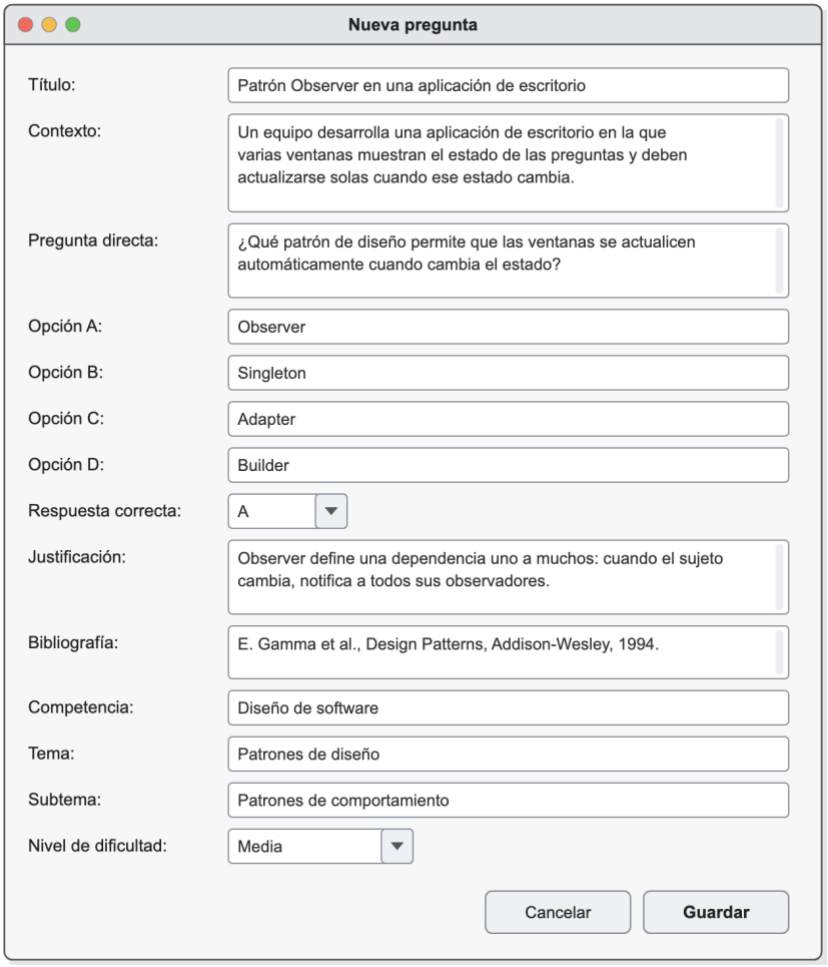


Figura 3.3 Prototipo del formulario «Nueva pregunta» (HU-01)

La Figura 3.4 corresponde a HU-02 y HU-03. En la parte superior están los filtros; la tabla muestra el estado con color y en la parte inferior están las acciones y la paginación.

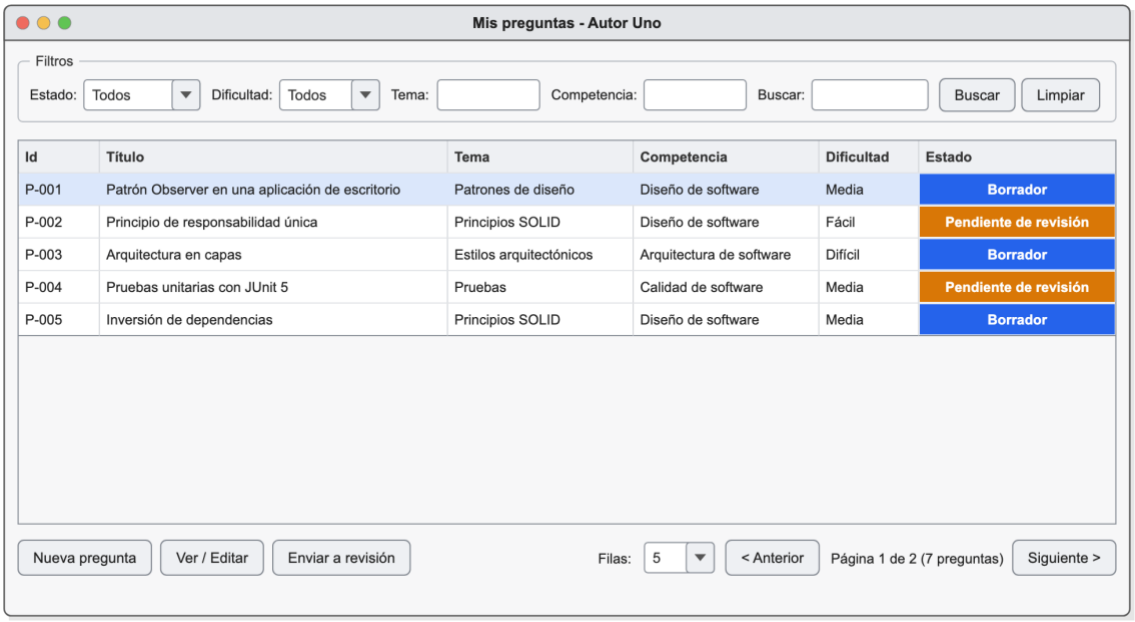


Figura 3.4 Prototipo de la ventana «Mis preguntas» con filtros, estados con color y paginación (HU-02 y HU-03)

La Figura 3.5 corresponde a HU-04. A la izquierda están las preguntas pendientes y a la derecha los revisores activos; el envío de correos se hace en segundo plano para no congelar la ventana.



Figura 3.5 Prototipo de la ventana «Asignar revisores» (HU-04)

3.2.- Test de usabilidad (thinking aloud)

Para evaluar la usabilidad se usa el método *thinking aloud* (pensar en voz alta) [8]: cada participante realiza las tareas de la Tabla 3.1 mientras dice lo que piensa, y un observador anota sus dudas, los errores y el tiempo, sin ayudarle. El test también verifica RNF-03: un usuario nuevo debe aprender a usar el sistema en menos de treinta minutos.

- **Participantes:** [Completar: número y perfil de los participantes, por ejemplo tres estudiantes que no participaron en el desarrollo]
- **Fecha y lugar:** [Completar]
- **Materiales:** aplicación en ejecución con los usuarios de prueba (autor1, admin, revisor1), guion de tareas y formato de observación.

#	Tarea	Criterio de éxito	Tiempo / éxito	Observaciones
1	Iniciar sesión como autor de preguntas.	Llega al tablero del autor.	[Completar]	[Completar]
2	Crear una pregunta completa.	Ve el mensaje «Pregunta P-00n guardada como Borrador».	[Completar]	[Completar]
3	Guardar una pregunta con dos opciones iguales.	Entiende el error y corrige la opción repetida.	[Completar]	[Completar]
4	Encontrar sus preguntas del tema «SOLID» en estado Borrador.	Usa los filtros y encuentra la pregunta.	[Completar]	[Completar]
5	Enviar una pregunta a revisión.	El estado cambia a «Pendiente de revisión» (ámbar).	[Completar]	[Completar]
6	Iniciar sesión como administrador y asignar dos revisores.	Ve «Revisores asignados y notificados por correo».	[Completar]	[Completar]

Tabla 3.1 Tareas del test de usabilidad

Hallazgos y mejoras: [Completar con los problemas observados durante el test y los cambios que hizo el equipo en la interfaz]

4.- Planificación del Sprint 1

El trabajo del primer corte se organizó como un sprint de Scrum. La Figura 4.1 muestra el tablero de tareas del sprint y la Tabla 4.1 resume las tareas que se derivaron de las historias de usuario.



Figura 4.1 Tablero de tareas del Sprint 1 [reemplazar por el pantallazo de Trello o Jira]

Tarea	Historia	Responsable	Estado
Diseñar los wireframes de las pantallas del corte.	HU-01 a HU-04	[Completar]	Terminada
Ampliar la entidad Question y la tabla questions con contexto, justificación, bibliografía, competencia, tema, subtema, dificultad y autor.	HU-01	[Completar]	Terminada
Implementar QuestionValidator (validación estructural) y sus pruebas.	HU-01	[Completar]	Terminada
Construir GUIQuestionForm y QuestionController.	HU-01	[Completar]	Terminada
Implementar el envío a revisión y los colores de estado (EstadoColores).	HU-02	[Completar]	Terminada
Implementar el listado con filtros y paginación (QuestionFilter, PageResult, GUIMyQuestions).	HU-03	[Completar]	Terminada
Crear la tabla review_assignments y SqliteReviewAssignmentRepository.	HU-04	[Completar]	Terminada
Implementar ReviewService, ReviewController y GUIAssignReviewers.	HU-04	[Completar]	Terminada

Tarea	Historia	Responsable	Estado
Implementar la notificación por correo (INotificador, EmailNotificador, ConsoleNotificador, NotificadorFactory).	HU-04	[Completar]	Terminada
Escribir las pruebas unitarias de dominio, controladores y repositorios.	Todas	[Completar]	Terminada
Elaborar el documento de arquitectura y el video de sustentación.	—	[Completar]	En curso

Tabla 4.1 Tareas del Sprint 1

5.- Requisitos no funcionales: escenarios de calidad

El atributo de calidad prioritario de esta iteración es la **modificabilidad** (RNF-10 a RNF-14). Cada escenario se especifica con el formato de Bass, Clements y Kazman [1]: contexto, fuente, estímulo, artefacto, entorno, respuesta, medida de la respuesta y resultado esperado.

5.1.- Escenario 1: nuevo canal de notificación

Contexto	Hoy el sistema avisa a los revisores por correo electrónico, o por consola si no hay configuración SMTP. El enunciado menciona otros canales posibles, como SMS o WhatsApp.
Fuente	Cliente o equipo de desarrollo.
Estímulo	Se pide notificar a los revisores por un canal nuevo (por ejemplo, WhatsApp).
Artefacto	Módulo de notificaciones (paquete infra) y ReviewService.
Entorno	Tiempo de diseño y desarrollo.
Respuesta	Se crea una clase que implementa INotificador y se agrega el caso en NotificadorFactory. ReviewService, los controladores y las vistas no cambian.
Medida de la respuesta	Una clase nueva y una clase modificada (NotificadorFactory); cero cambios en los paquetes domain, controller y presentation; menos de 4 horas-persona; las 110 pruebas existentes siguen pasando.
Resultado esperado	El cambio queda aislado gracias a la abstracción INotificador, la fábrica y la inyección de dependencias en AppContext.

Tabla 5.1 Escenario de calidad de modificabilidad 1

5.2.- Escenario 2: nuevo tipo de pregunta

Contexto	RNF-13 pide incorporar nuevos tipos de preguntas sin cambios significativos. El generador de preguntas usa un microkernel con plugins registrados en plugins.properties.
Fuente	Cliente (comité del programa).
Estímulo	Se pide un tipo de pregunta nuevo, por ejemplo basada en una imagen o de verdadero/falso.
Artefacto	Microkernel de preguntas (paquetes kernel, plugin.api, plugin.manager y plugins).
Entorno	Tiempo de desarrollo y configuración.
Respuesta	Se crea una clase que implementa QuestionPlugin, con su propio pipeline de filtros si lo necesita, y se agrega la línea plugin.<nombre>=<clase> en plugins.properties. QuestionMicrokernel y QuestionPluginManager no cambian.
Medida de la respuesta	Una clase nueva (más sus filtros) y una línea de configuración; cero líneas modificadas en el núcleo.
Resultado esperado	El sistema crece por extensión, como pide el principio abierto/cerrado.

Tabla 5.2 Escenario de calidad de modificabilidad 2

5.3.- Escenario 3: cambio del motor de base de datos

Contexto	En el primer corte la persistencia usa SQLite por simplicidad; el enunciado sugiere MariaDB o PostgreSQL para los siguientes cortes.
Fuente	Equipo de desarrollo.
Estímulo	Se decide migrar la persistencia a PostgreSQL.
Artefacto	Capa de acceso a datos (paquete access) y raíz de composición (AppContext).
Entorno	Tiempo de desarrollo.
Respuesta	Se implementan nuevos repositorios que cumplen QuestionRepository, ReviewAssignmentRepository e IUserRepository, y se cambian las instancias que crea AppContext. Servicios, controladores y vistas no se modifican.
Medida de la respuesta	Tres clases nuevas y una clase modificada (AppContext); cero cambios en domain, controller y presentation; las pruebas de servicios, que usan repositorios falsos, pasan sin cambios.
Resultado esperado	La capa de dominio es independiente del motor de base de datos (patrón Repository e inversión de dependencias).

Tabla 5.3 Escenario de calidad de modificabilidad 3

6.- Decisiones de arquitectura

6.1.- Tipo de aplicación: escritorio

Se construyó una **aplicación de escritorio** en Java porque el entregable del primer corte lo exige y porque los usuarios (autores, revisores, docentes y administradores del programa) trabajan desde equipos institucionales.

- **Ventajas:** se ejecuta localmente sin servidor; Swing ofrece una interfaz rica (tablas, listas, colores por estado); el despliegue es sencillo (la aplicación y un archivo de base de datos).
- **Desventajas:** cada usuario instala la aplicación y la base de datos SQLite es local, por lo que los datos no se comparten entre equipos. Esto se resolverá en el segundo corte con la arquitectura distribuida de microservicios.

6.2.- Estilo arquitectónico: monolito en tres capas con MVC

La aplicación es un monolito organizado en tres capas. Cada capa usa solo la capa inferior, y lo hace a través de interfaces:

1. **Presentación:** vistas Swing (paquete presentation) y controladores (paquete controller).
2. **Dominio:** entidades, servicios, validadores e interfaces de repositorio y notificación (paquete domain).
3. **Acceso a datos e infraestructura:** repositorios SQLite (paquete access) y notificadores de correo (paquete infra).

Dentro de la presentación se usa el micropatrón **MVC** [6]: el modelo son las entidades y los servicios de dominio, las vistas son las ventanas Swing y los controladores (QuestionController y ReviewController) reciben las acciones del usuario y llaman a los servicios. Las vistas se enteran de los cambios del modelo con el patrón **Observer**.

Este estilo se eligió por las siguientes razones:

- Es el estilo pedido para el primer corte.
- Separa la lógica de negocio, la interfaz y la persistencia (RNF-12).

- Permite probar el dominio sin interfaz ni base de datos.
- Facilita la modificabilidad (capítulo 5).
- Prepara la evolución hacia microservicios: los servicios de dominio ya están separados por contexto (usuarios, preguntas y revisión).

El generador de preguntas usa además los estilos **Microkernel** y **Tuberías y filtros** [6] (sección 9.2).

6.3.- Selección de tecnologías

Tecnología	Uso en el proyecto	Justificación, ventajas y desventajas
Java 17	Lenguaje de programación (maven.compiler.release 17).	Sugerido por el curso; orientado a objetos, tipado estático y multiplataforma. Desventaja: más verboso que otros lenguajes.
Java Swing	Interfaz gráfica de escritorio.	Incluido en el JDK, sin dependencias extra y sugerido por el enunciado. Desventaja: apariencia menos moderna que JavaFX.
Apache Maven	Construcción, dependencias y ejecución (exec-maven-plugin).	Estándar en Java; el proyecto se compila, se prueba y se ejecuta con un solo comando.
SQLite (sqlite-jdbc 3.45.1.0)	Persistencia embebida en el archivo users.db.	No requiere servidor ni instalación y permite bases en memoria para las pruebas. Desventaja: un solo archivo local y poca concurrencia; por eso se planea migrar (escenario 5.3).
argon2-jvm 2.11 (Argon2id)	Hash de contraseñas.	Cumple RNF-08: Argon2 ganó la Password Hashing Competition y su variante Argon2id resiste ataques con GPU [9]. Desventaja: usa una biblioteca nativa.
Jakarta Mail 2.0.1	Envío de correos SMTP con STARTTLS.	API estándar de correo para Java.
JUnit 5.10.1	Pruebas unitarias automatizadas.	Estándar de facto; permite nombres descriptivos (@DisplayName) y se integra con Maven.
Git y GitHub	Control de versiones y trabajo colaborativo.	Requeridos por el curso.
[Completar: Trello o Jira]	Gestión del sprint.	Tablero de tareas del equipo (Figura 4.1).

Tabla 6.1 Tecnologías seleccionadas

Las respuestas a las preguntas del enunciado son:

- **¿Qué tecnologías ayudan a implementar el estilo arquitectónico?** Los paquetes de Java separan las capas, y las interfaces de Java permiten la

inversión de dependencias entre ellas. Swing implementa la vista del MVC, y el soporte de reflexión de Java permite cargar los plugins del microkernel desde plugins.properties.

- **¿Qué tecnologías ayudan a implementar el tipo de aplicación?** Java Swing y SQLite permiten una aplicación de escritorio autónoma, y Maven la empaqueta y la ejecuta.
- **¿Qué tecnologías ayudan a cumplir los requisitos no funcionales?** Argon2id cubre RNF-08 (contraseñas con hash seguro). JUnit 5 sostiene la modificabilidad, porque cada cambio se valida con pruebas automáticas. Jakarta Mail queda aislado detrás de INotificador, lo que también apoya la modificabilidad.

7.- Arquitectura del sistema

La arquitectura se documenta con el modelo C4 de Simon Brown [3]: contexto, contenedores, componentes y código. El nivel de código se representa con diagramas de clases UML. Como vista de procesos del modelo 4+1 [7] se incluyen dos diagramas de secuencia de las historias principales.

7.1.- Nivel 1: diagrama de contexto

El sistema interactúa con cinco tipos de usuario y con un sistema externo: el servidor de correo SMTP, que entrega las notificaciones de asignación. En este corte el autor, el administrador y el revisor tienen funciones implementadas; las del docente y el estudiante (reportes y simulacros) llegan en los próximos cortes.

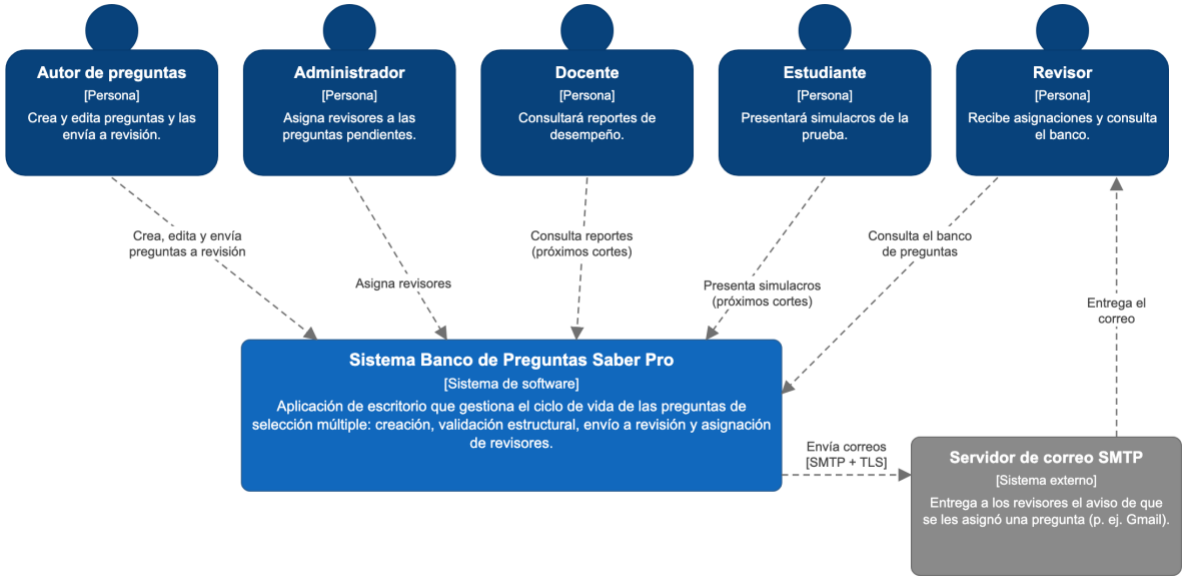


Figura 7.1 Diagrama de contexto (C4, nivel 1)

7.2.- Nivel 2: diagrama de contenedores

El sistema tiene tres contenedores:

- La aplicación de escritorio, que es el monolito Java.
- La base de datos SQLite, guardada en el archivo users.db.
- Los archivos de configuración: plugins.properties registra los plugins y mail.properties guarda las credenciales SMTP. Este último está en el .gitignore y nunca se sube al repositorio.

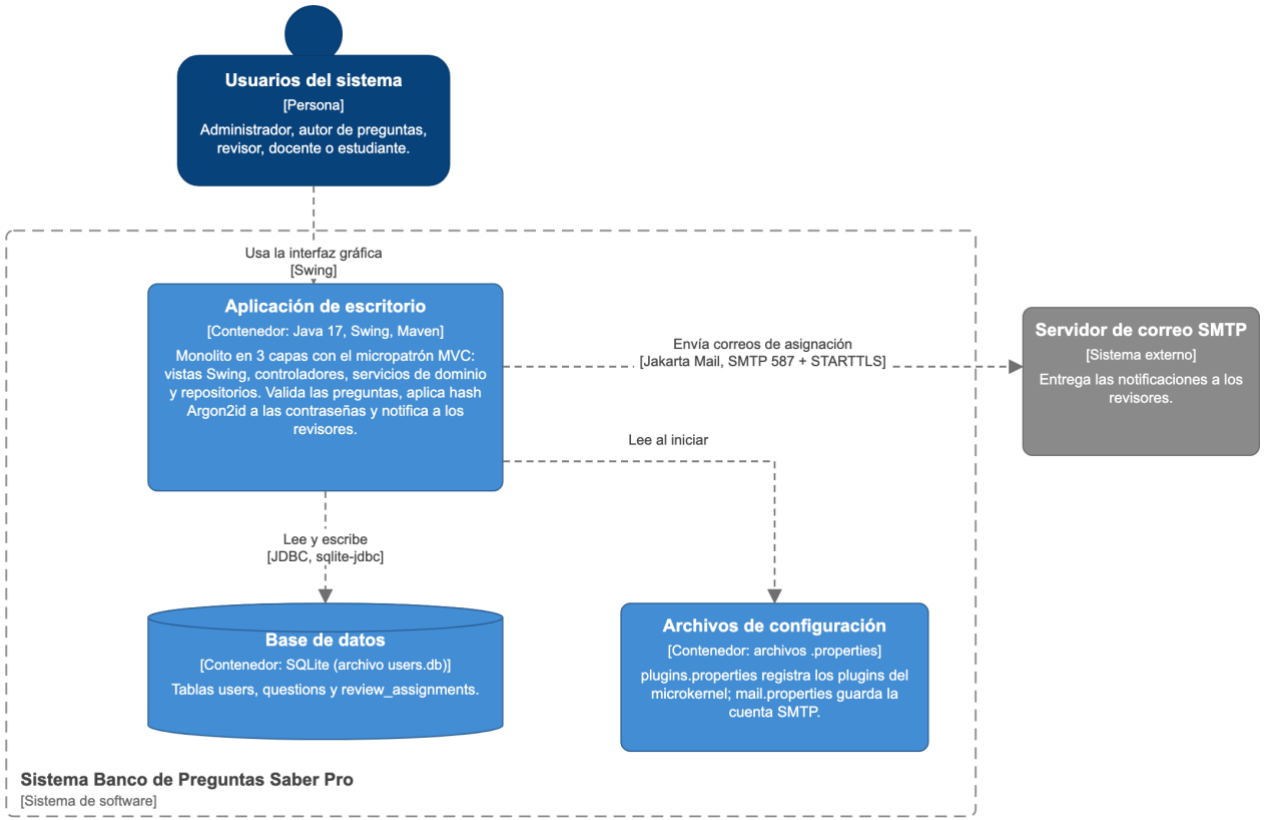


Figura 7.2 Diagrama de contenedores (C4, nivel 2)

7.3.- Nivel 3: diagrama de componentes

La Figura 7.3 descompone la aplicación de escritorio en sus componentes. Las flechas verticales muestran el flujo de las capas: las vistas usan los controladores, los controladores invocan los servicios y los servicios persisten mediante repositorios. AppContext es la raíz de composición: crea todos los objetos concretos y los inyecta, de modo que ningún servicio conoce las clases de SQLite ni de correo.

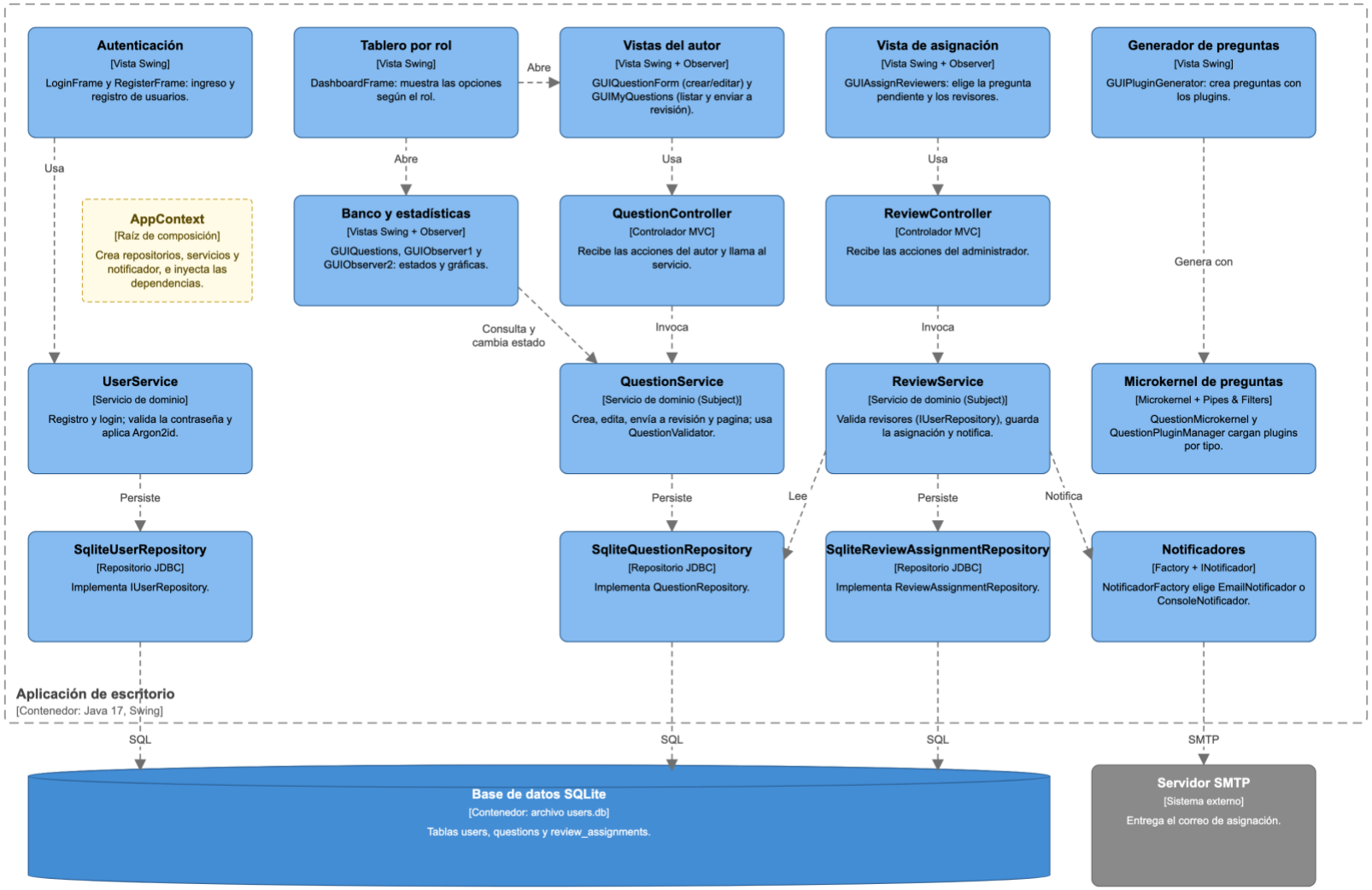


Figura 7.3 Diagrama de componentes de la aplicación de escritorio (C4, nivel 3)

La Tabla 7.1 relaciona cada paquete del código con su capa.

Paquete	Clases principales	Capa / responsabilidad
presentation	LoginFrame, RegisterFrame, DashboardFrame, GUIQuestionForm, GUIMyQuestions, GUIAssignReviewers, GUIQuestions, GUIObserver1, GUIObserver2, GUIPluginGenerator, EstadoColores	Presentación (vista)
controller	QuestionController, ReviewController	Presentación (controlador)
domain	Entidades (Question, QuestionDistractors, User, Role, UserStatus, ReviewAssignment); servicios (QuestionService, ReviewService, UserService); QuestionValidator, PasswordValidator, PasswordHasher, QuestionFilter, PageResult; interfaces QuestionRepository, ReviewAssignmentRepository, INotificador, IPasswordHasher, IPasswordValidator	Dominio (modelo)
access	SqliteDatabase, SqliteUserRepository, SqliteQuestionRepository, SqliteReviewAssignmentRepository, QuestionImplRepository (en memoria), IUserRepository	Acceso a datos
infra	Subject, Observer, NotificadorFactory, EmailNotificador, ConsoleNotificador	Infraestructura
kernel, plugin.api, plugin.manager, plugins, pipeline	QuestionMicrokernel, QuestionPlugin, QuestionRequest, QuestionPluginManager, plugins de selección múltiple, de caso y multimedia, Filter<T>, Pipeline<T> y sus filtros	Generador de preguntas (Microkernel y Tuberías y filtros)
co.edu.unicauca	Main, AppContext	Arranque y raíz de composición

Tabla 7.1 Paquetes del código y su responsabilidad

7.4.- Nivel 4: diagramas de clases

La Figura 7.4 muestra las clases del dominio. Question contiene sus cuatro opciones en QuestionDistractors; ReviewAssignment referencia la pregunta y a los usuarios por su identificador.

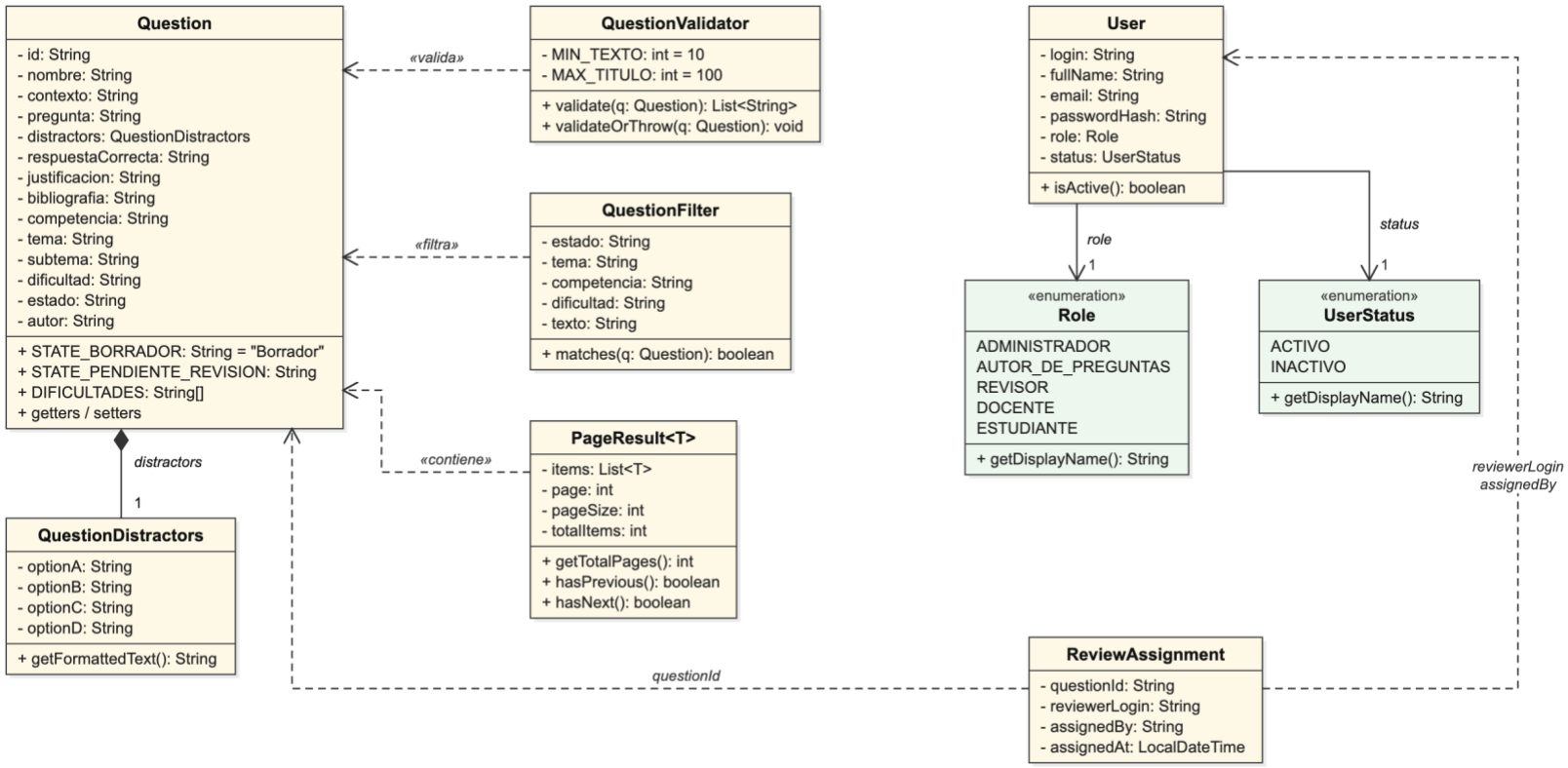


Figura 7.4 Diagrama de clases del dominio

La Figura 7.5 muestra el MVC con el patrón Observer. Las vistas usan los controladores, los controladores delegan en los servicios, y los servicios heredan de Subject para notificar a las vistas que implementan Observer.

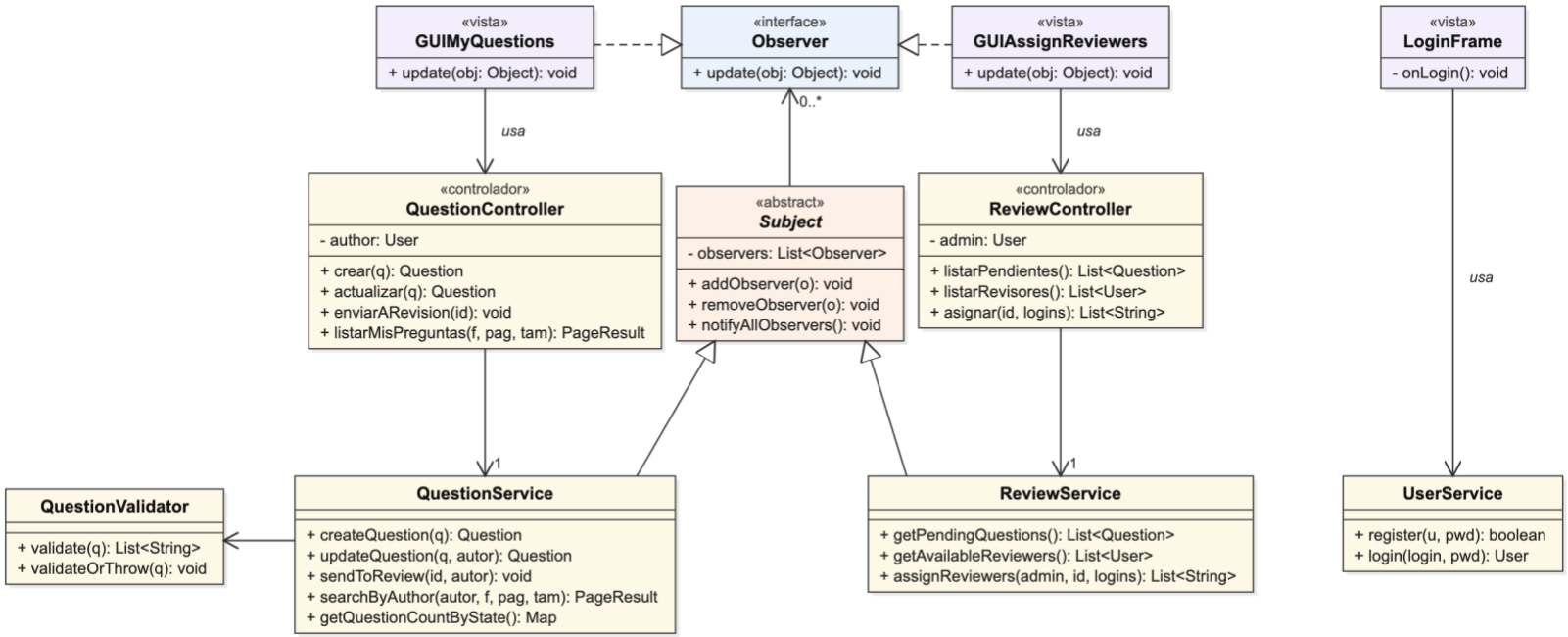


Figura 7.5 Diagrama de clases de la presentación, los controladores y los servicios (MVC y Observer)

La Figura 7.6 muestra la inversión de dependencias. Los servicios dependen solo de interfaces, y las implementaciones concretas viven en la capa de acceso a datos.

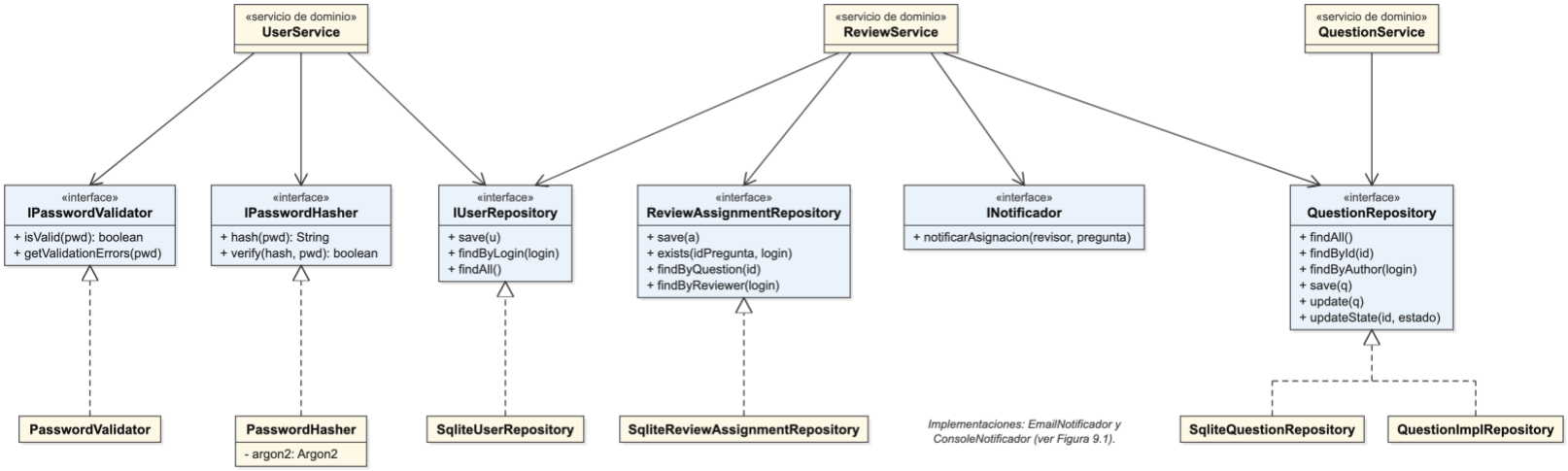


Figura 7.6 Diagrama de clases de la inversión de dependencias entre servicios, interfaces y repositorios

7.5.- Vista dinámica: diagramas de secuencia

La Figura 7.7 muestra HU-01. La vista entrega la pregunta al controlador, que le asigna el autor. El servicio aplica la validación estructural: si hay errores, la excepción con todos los mensajes vuelve a la vista; si no hay errores, se asigna el id, se guarda en estado «Borrador» y se notifica a los observadores.

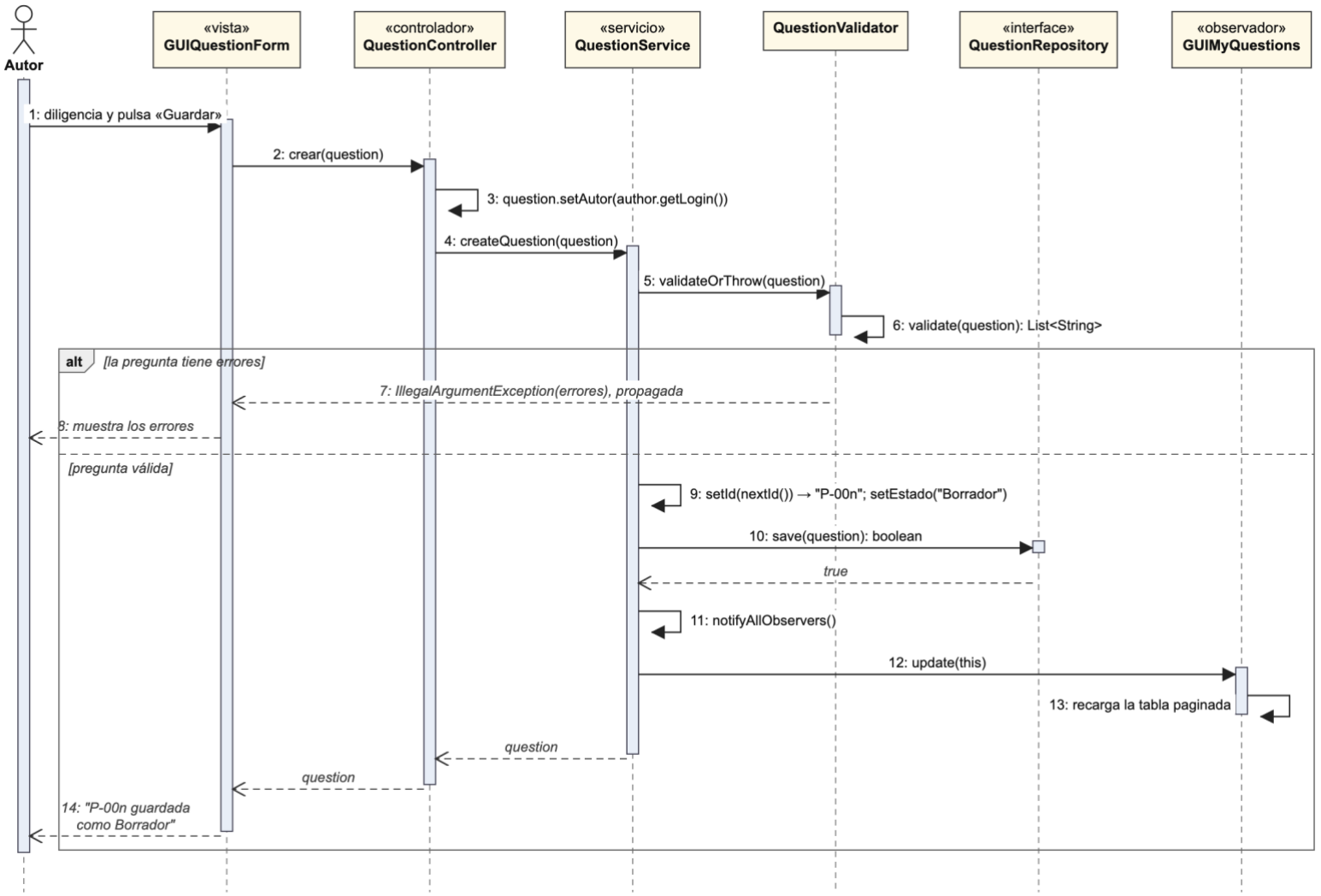


Figura 7.7 Diagrama de secuencia de HU-01: crear pregunta

La Figura 7.8 muestra HU-04. La asignación se ejecuta con un SwingWorker para que el envío de correos no bloquee la interfaz. El servicio valida a todos los revisores antes de guardar, para no dejar asignaciones a medias. Después

guarda cada asignación nueva y notifica al revisor; si un correo falla, la asignación se conserva y el fallo se informa al administrador.

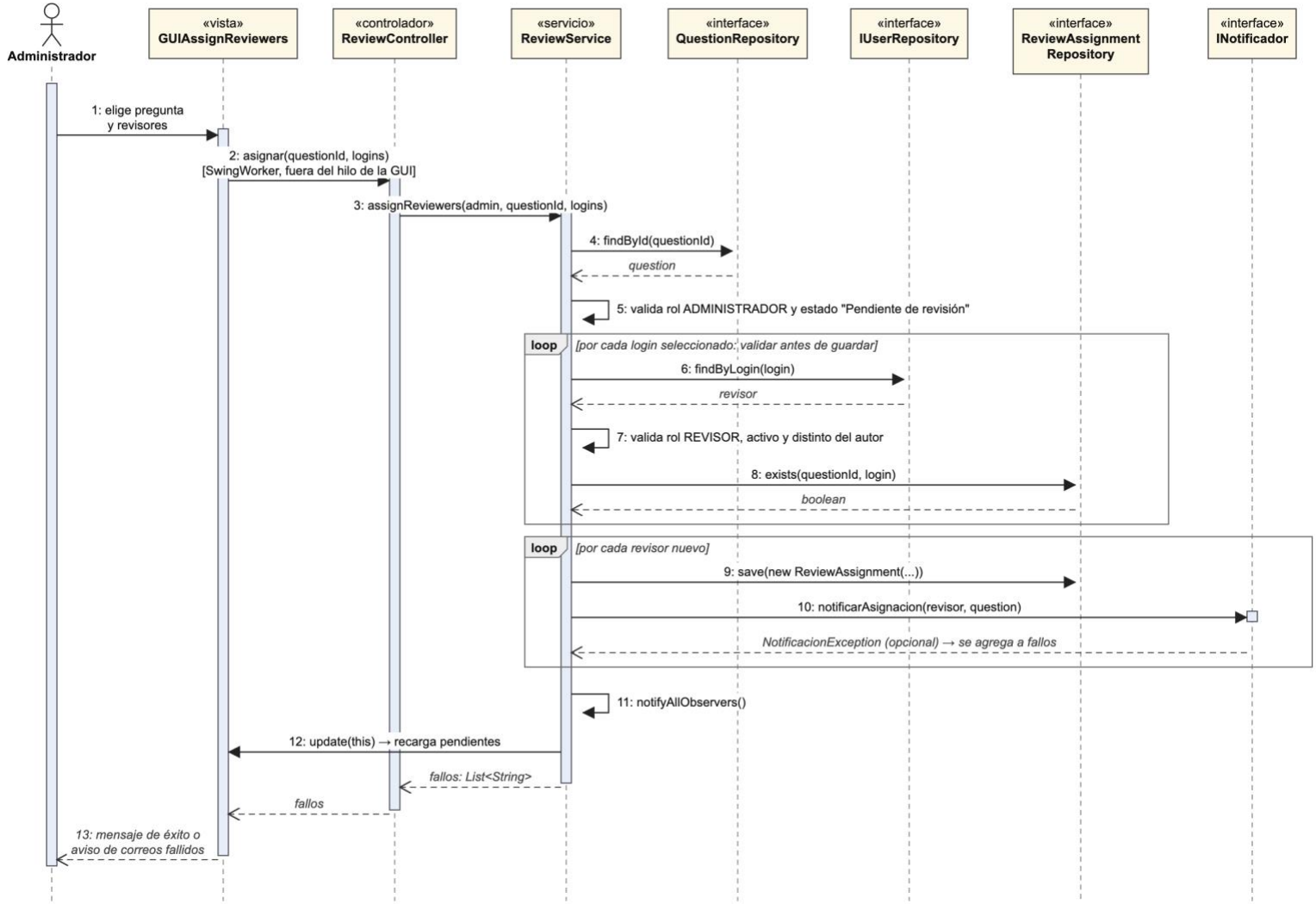


Figura 7.8 Diagrama de secuencia de HU-04: asignar revisores

8.- Modelo de datos

La Figura 8.1 presenta el modelo entidad-relación de todo el sistema, elaborado con la notación de PowerDesigner.

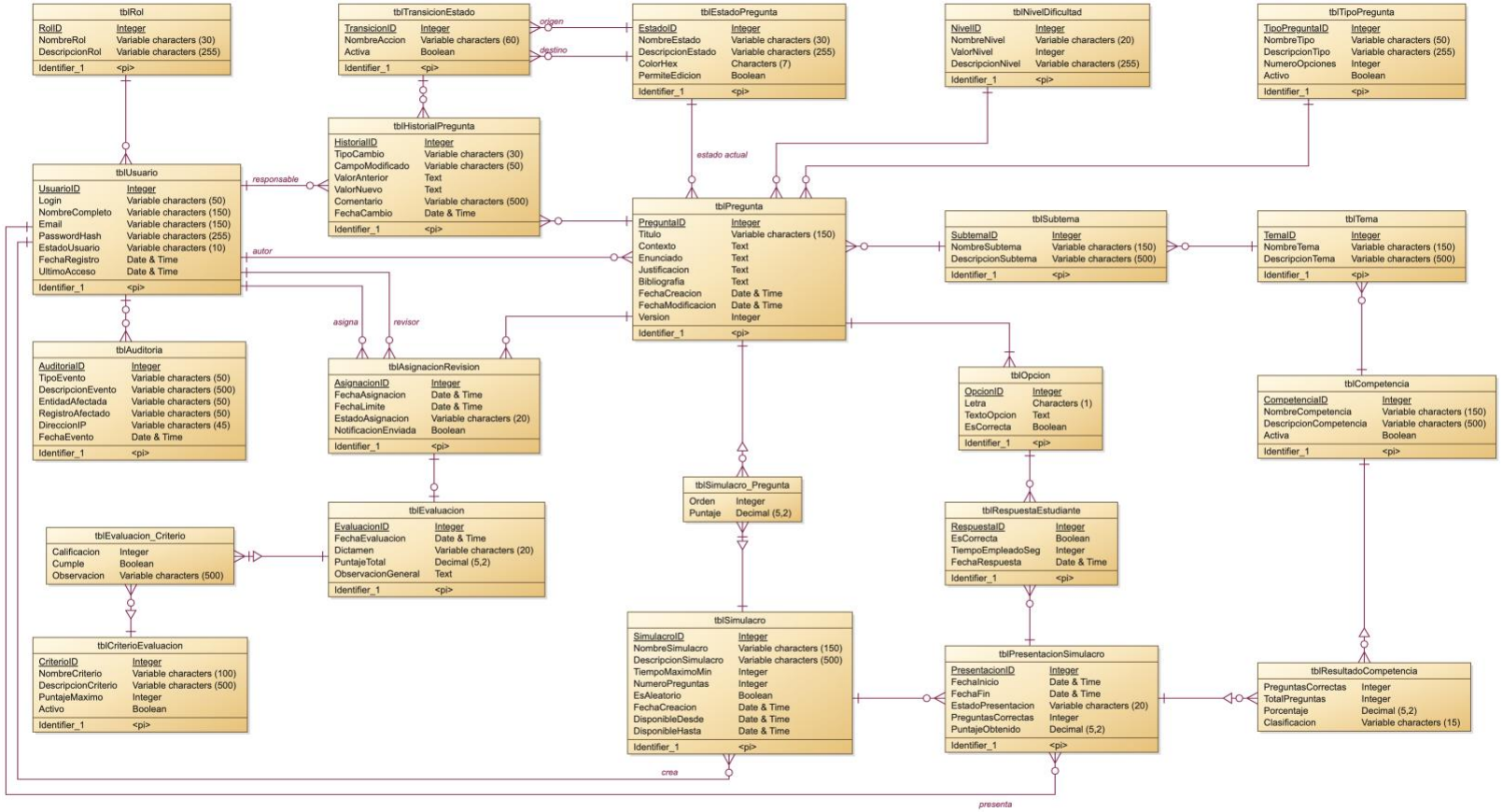


Figura 8.1 Modelo entidad-relación del sistema Banco de Preguntas Saber Pro

El modelo tiene 22 entidades y 30 relaciones y cubre las ocho historias épicas del enunciado: usuarios y roles, banco de preguntas, validación, ciclo de vida, revisión por pares, simulacros, seguimiento académico y trazabilidad. Las

entidades sin identificador propio (marcadas con un triángulo) son dependientes y se identifican por sus entidades padre.

En el primer corte la base SQLite implementa el subconjunto que necesitan las historias del corte (Tabla 8.1). Las tablas catálogo del modelo (competencia, tema, subtema, nivel, estado y tipo de pregunta) y las entidades de revisión, simulacros y seguimiento se incorporarán en los próximos cortes.

Tabla SQLite actual	Columnas	Entidades del modelo
users	login (PK), full_name, role, status, password_hash, email	tblUsuario, tblRol
questions	id (PK), nombre, contexto, pregunta, option_a a option_d, respuesta_correcta, justificacion, bibliografia, competencia, tema, subtema, dificultad, estado, autor	tblPregunta, tblOpcion y catálogos de clasificación
review_assignments	question_id + reviewer_login (PK compuesta), assigned_by, assigned_at	tblAsignacionRevision

Tabla 8.1 Tablas implementadas en el primer corte y su correspondencia con el modelo

9.- Principios SOLID y patrones de diseño

9.1.- Principios SOLID

Principio	Cómo se aplica en el código
Responsabilidad única (SRP)	Cada clase tiene una sola razón para cambiar. QuestionValidator solo valida la estructura; PasswordValidator solo revisa la complejidad de la contraseña; PasswordHasher solo aplica Argon2id; los repositorios solo persisten; los controladores solo traducen acciones de la vista; EstadoColores solo decide el color de cada estado.
Abierto/cerrado (OCP)	Se agregan comportamientos nuevos sin modificar el código existente: nuevos notificadores (INotificador), nuevos repositorios (interfaces de repositorio) o nuevos tipos de pregunta (plugins del microkernel registrados en plugins.properties).
Sustitución de Liskov (LSP)	SqliteQuestionRepository y QuestionImplRepository son intercambiables bajo QuestionRepository, igual que EmailNotificador y ConsoleNotificador bajo INotificador. En las pruebas los servicios funcionan sin cambios con FakeUserRepository, FakeAssignmentRepository y FakeNotificador.
Segregación de interfaces (ISP)	Las interfaces son pequeñas y específicas: INotificador tiene un método; IPasswordHasher e IPasswordValidator, dos; IUserRepository, tres. Cada cliente depende solo de lo que usa.
Inversión de dependencias (DIP)	UserService, QuestionService y ReviewService dependen de abstracciones que reciben por el constructor. AppContext es el único lugar donde se crean las clases concretas (Figura 7.6).

Tabla 9.1 Aplicación de los principios SOLID

9.2.- Patrones de diseño implementados

Patrón	Tipo	Clases	Problema que resuelve y cómo se aplicó
Modelo-Vista-Controlador	Micropatrón arquitectónico	Vistas Swing; QuestionController y ReviewController; servicios y entidades de dominio	Separa la interfaz de la lógica: las vistas solo hablan con los controladores, y estos con los servicios (Figura 7.5).
Observer	GoF, de comportamiento	Subject, Observer; QuestionService y ReviewService (sujetos); GUIMyQuestions, GUIAssignReviewers, GUIObserver1 y GUIObserver2 (observadores)	Varias ventanas deben refrescarse cuando cambia una pregunta, sin que el servicio conozca las vistas. AppContext encadena los sujetos: un cambio en QuestionService hace que ReviewService avise a su vista.

Patrón	Tipo	Clases	Problema que resuelve y cómo se aplicó
Factory (fábrica simple)	Creacional	NotificadorFactory, EmailNotificador, ConsoleNotificador	Decide qué notificador crear: correo real si mail.properties está completo y simulación por consola en otro caso. El resto del sistema solo conoce INotificador (Figura 9.1).
Strategy	GoF, de comportamiento	INotificador, IPasswordHasher, IPasswordValidator y sus implementaciones	Algoritmos intercambiables (cómo notificar, cómo hacer hash, cómo validar la contraseña) que se inyectan en los servicios.
Singleton	GoF, creacional	QuestionPluginManager.getInstance()	Garantiza un único registro de plugins, cargado una sola vez al iniciar la aplicación.
Repository	Patrón de arquitectura empresarial [5]	QuestionRepository, ReviewAssignmentRepository, IUserRepository y sus implementaciones SQLite	Aísla el dominio del motor de base de datos (escenario 5.3).
Inyección de dependencias (raíz de composición)	Patrón de diseño	AppContext	Centraliza la creación de objetos concretos; los servicios reciben sus dependencias por el constructor.
Microkernel	Estilo arquitectónico [6]	QuestionMicrokernel, QuestionPluginManager, QuestionPlugin y los plugins de selección múltiple, de caso y multimedia	Permite agregar tipos de pregunta como plugins sin tocar el núcleo (RNF-13, escenario 5.2, Figura 9.2).
Tuberías y filtros	Estilo arquitectónico [6]	Pipeline<T>, Filter<T>, ContentValidationFilter, OptionsValidationFilter, ClassificationFilter, CorrectAnswerValidationFilter	La pregunta de selección múltiple pasa por cuatro filtros independientes y reutilizables: contenido, opciones, clasificación y respuesta correcta.

Tabla 9.2 Patrones de diseño implementados

La Figura 9.1 detalla el patrón Factory de los notificadores y la Figura 9.2 el microkernel con su pipeline.

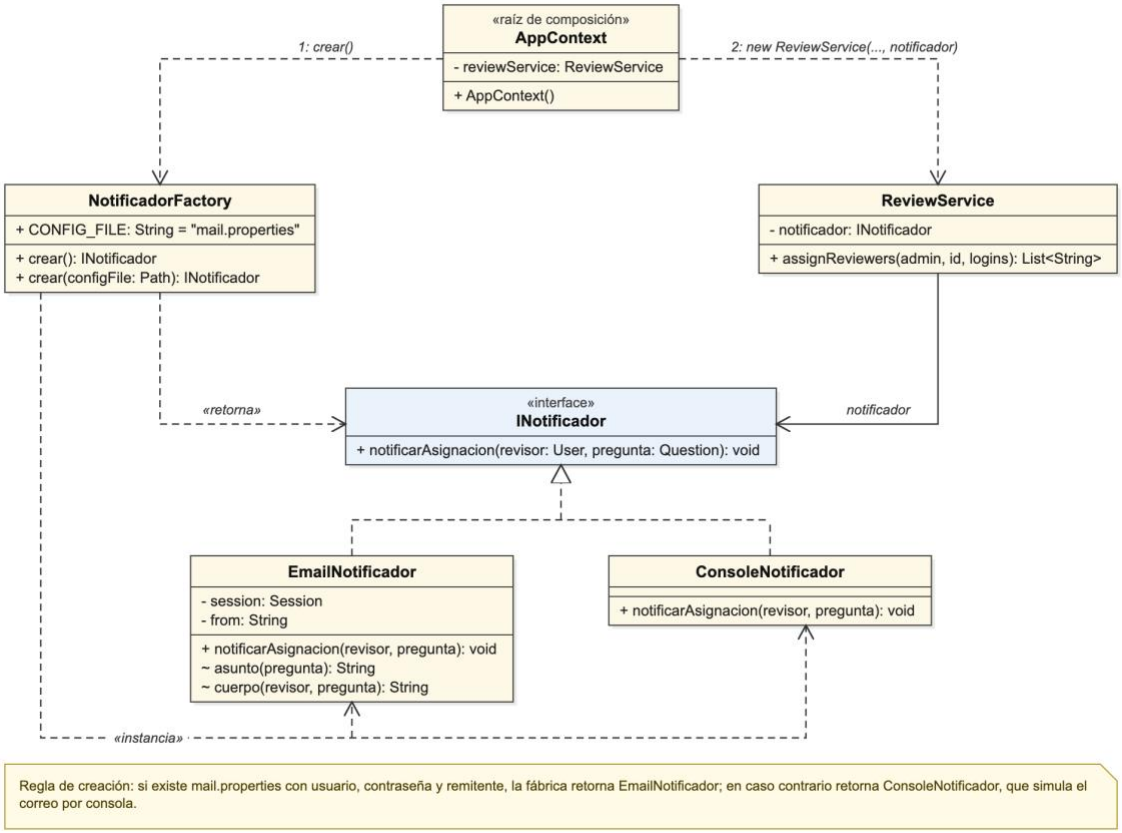


Figura 9.1 Patrón Factory para crear el notificador de revisores

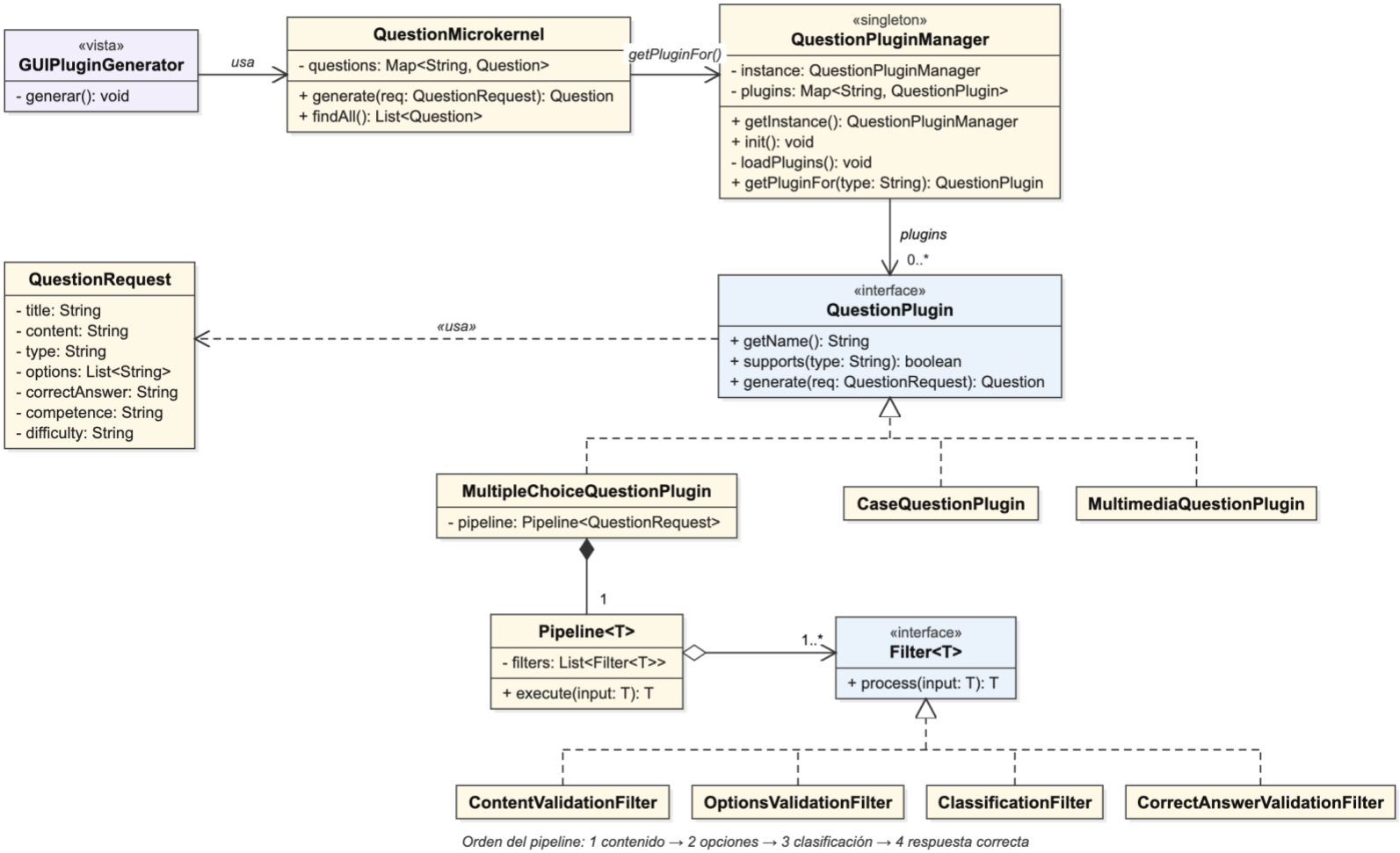


Figura 9.2 Microkernel de preguntas con tuberías y filtros

9.3.- Oportunidades de mejora para el segundo corte

La revisión del código dejó estas mejoras para la siguiente iteración:

- Mover IUserRepository del paquete access al paquete domain, como las demás interfaces de repositorio, para que el dominio no dependa de la capa de acceso a datos.
- Inyectar QuestionValidator en QuestionService en lugar de crearlo con new, y agregar las reglas de RF-12 y RF-13.
- Mover PasswordHasher al paquete infra, porque depende de una biblioteca externa (Argon2).
- Representar los estados de la pregunta con un enum o con el patrón State, para controlar todas las transiciones válidas de RF-14 y RF-15.
- Inyectar QuestionPluginManager en lugar de usar su instancia estática, para facilitar las pruebas.

10.- Pruebas unitarias automatizadas

Las pruebas se escribieron con JUnit 5 y se ejecutan con `mvn test`. Hay 18 clases de prueba con 110 casos (@Test). Los servicios se prueban con repositorios y notificadores falsos (FakeUserRepository, FakeAssignmentRepository y FakeNotificador), así que no necesitan base de datos ni correo. Los repositorios SQLite se prueban con una base de datos en memoria (jdbc:sqlite::memory:).

Clase de prueba	Qué verifica	Casos
QuestionServiceHistoriasTest	HU-01 a HU-03: creación en Borrador, edición solo del autor, envío a revisión, filtros y paginación.	20
ReviewServiceTest	HU-04: pendientes, revisores válidos, asignación múltiple sin duplicados, fallos de correo y permisos del administrador.	14
QuestionValidatorTest	Validación estructural: campos obligatorios, longitudes, opciones repetidas, respuesta y dificultad.	10
UserServiceTest	Registro y autenticación de usuarios.	8
SqliteQuestionRepositoryTest	Guardar, buscar, listar por autor y actualizar preguntas en SQLite.	8
QuestionServiceTest	Consultas, cambio de estado y estadísticas por estado.	6
PasswordValidatorTest	Reglas de complejidad de la contraseña.	5
SqliteUserRepositoryTest	Persistencia de usuarios y correo.	5
SqliteReviewAssignmentRepository Test	Persistencia de asignaciones y unicidad pregunta-revisor.	5
QuestionFilterTest	Filtros por estado, dificultad, tema, competencia y texto.	4
QuestionTest	Entidad Question y formato de las opciones.	4
UserTest	Entidad User, roles y estados.	4
MultipleChoicePipelineTest	Pipeline de filtros del plugin de selección múltiple.	4
PageResultTest	Cálculo de páginas y navegación.	3
PasswordHasherTest	Hash y verificación con Argon2id.	3
QuestionImplRepositoryTest	Repositorio en memoria.	3
ControllersTest	Controladores y flujo completo autor → administrador.	2
ObserverPatternTest	Notificación y retiro de observadores.	2
Total		**110**

Tabla 10.1 Pruebas unitarias automatizadas

Evidencia de ejecución: [Completar: pantallazo del resultado de mvn test]

11.- Enlaces del proyecto

- **Video de sustentación (YouTube):** [Completar con la URL del video]
- **Repositorio GitHub:** [Completar con la URL del repositorio]

Para compilar, probar y ejecutar la aplicación:

- Compilar: `mvn compile`
- Ejecutar las pruebas: `mvn test`
- Ejecutar la aplicación: `mvn exec:java`

Cuando la base de datos está vacía, la aplicación crea los usuarios de prueba admin, autor1, revisor1, docente1 y estudiante1; sus contraseñas están en el README del repositorio. Para enviar correos reales se copia mail.properties.example como mail.properties y se completan los datos SMTP; sin ese archivo los correos se simulan por consola.

Bibliografía

[1] L. Bass, P. Clements y R. Kazman, *Software Architecture in Practice*, 4.ª ed. Boston, MA, EE. UU.: Addison-Wesley, 2021.

[2] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, EE. UU.: Addison-Wesley, 1994.

[3] S. Brown, "The C4 model for visualising software architecture". [En línea]. Disponible en: https://c4model.com

[4] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, EE. UU.: Prentice Hall, 2017.

[5] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, EE. UU.: Addison-Wesley, 2002.

[6] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad y M. Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Chichester, Reino Unido: Wiley, 1996.

[7] P. B. Kruchten, "The 4+1 View Model of architecture," *IEEE Software*, vol. 12, n.º 6, pp. 42–50, 1995.

[8] J. Nielsen, *Usability Engineering*. San Francisco, CA, EE. UU.: Morgan Kaufmann, 1993.

[9] A. Biryukov, D. Dinu y D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," en *Proc. IEEE European Symposium on Security and Privacy*, 2016, pp. 292–302.

[10] Universidad del Cauca, "Proyecto de clase 2026.2: Sistema para la gestión, validación y administración de un banco de preguntas para la preparación de las pruebas Saber Pro," Ingeniería de Software II, 2026.